

Jacob Probst
MLOPs Final Project/HW 5
SIES 765
https://github.com/prob7276/MLOPS_Final_Project

1. Describe your system end-to-end

This system includes the training flow and the inference flow managed by Apache Airflow. The training flow orchestrates an upload and training of a breast cancer dataset. This stores a model.pkl in AWS S3. The inference flow takes that dataset for an SQS queue and sends a message per record. Finally, the consumption layer utilizes a Docker container and Kubernetes. This section downloads the S3 model, checks the SQS queue, runs inferences on those messages, and stores the prediction back onto the same S3 bucket as a .json file.

2. Why is a queue used instead of direct API calls?

The overall main reason to use a queue instead of direct API calls is to protect the data. The data is protected in various ways, include via the message queue's ability to handle a large-scale increase in traffic. An API may crash with a spike in traffic, but a message queue will just wait for workers to become available. The system is also decoupled between producers and consumers. This means that the product is not affected if one of the consumers are down. Lastly, if all consumers are down, the messages would remain in the queue rather than being deleted.

3. What happens if a consumer crashes mid-processing?

If a consumer crashes mid-processing, the SQS process will ensure "at-least-once" delivery. This means that the message will remain in the SQS queue and will not be deleted from the queue until the message is written to the S3 location. This ensures data integrity, meaning no messages are lost because of an outage.

4. Where are the bottlenecks in your system?

There are bottlenecks in the S3 writing. Each instance is saving to the same location. To reduce the bottleneck, it would be wise to create separate locations to save predictions for each model. Additionally, the model could quickly become a bottleneck as the stop and start for each prediction initiation would begin to show itself at scale. When the system is put into production, hosting the model in a container may result in a faster system.

5. One improvement you would make for production

Two improvements I would make to my system are automatic scaling and corrections for data drift. Currently, this system is only scaled on command. To better automate the system, enabling auto-scaling would allow the system to run more efficiently during spikes, and save on compute during lulls. Additionally, to address concerns of data drift as the dataset changes, implementing a closed-loop model would create a better outcome in production. This would include an ability to monitor ground-truth, implementing feedback from real-life results, and updating the training model to train on more recent data and feedback rather than the whole dataset.